

Introduction à la parallélisation

Erwan Cossevin (GMAP), d'après le support de Niels Oger (DSI)
Formation initiale TS - 02/2025

Objectifs du cours

- Comprendre comment la parallélisation exploite les ressources d'une machine.
- Comprendre les enjeux liés à la parallélisation (gains et limites) et savoir identifier quand il est pertinent de l'utiliser.
- Avoir des exemples simples de parallélisation.

Les TP se déroulent sous Linux.

Pour commencer, lancer dans un terminal :

```
git clone http://gitlab.meteo.fr/tutoriel-hpc/introduction-parallelisation.git
```

Plan de la présentation

- 1) Hardware : Processeur et mémoire(s)
- 2) Software : Processus et thread
- 3) Parallélisation sur un seul nœud
- 4) Communication entre processus et MPI
- 5) Optimiser avant de paralléliser

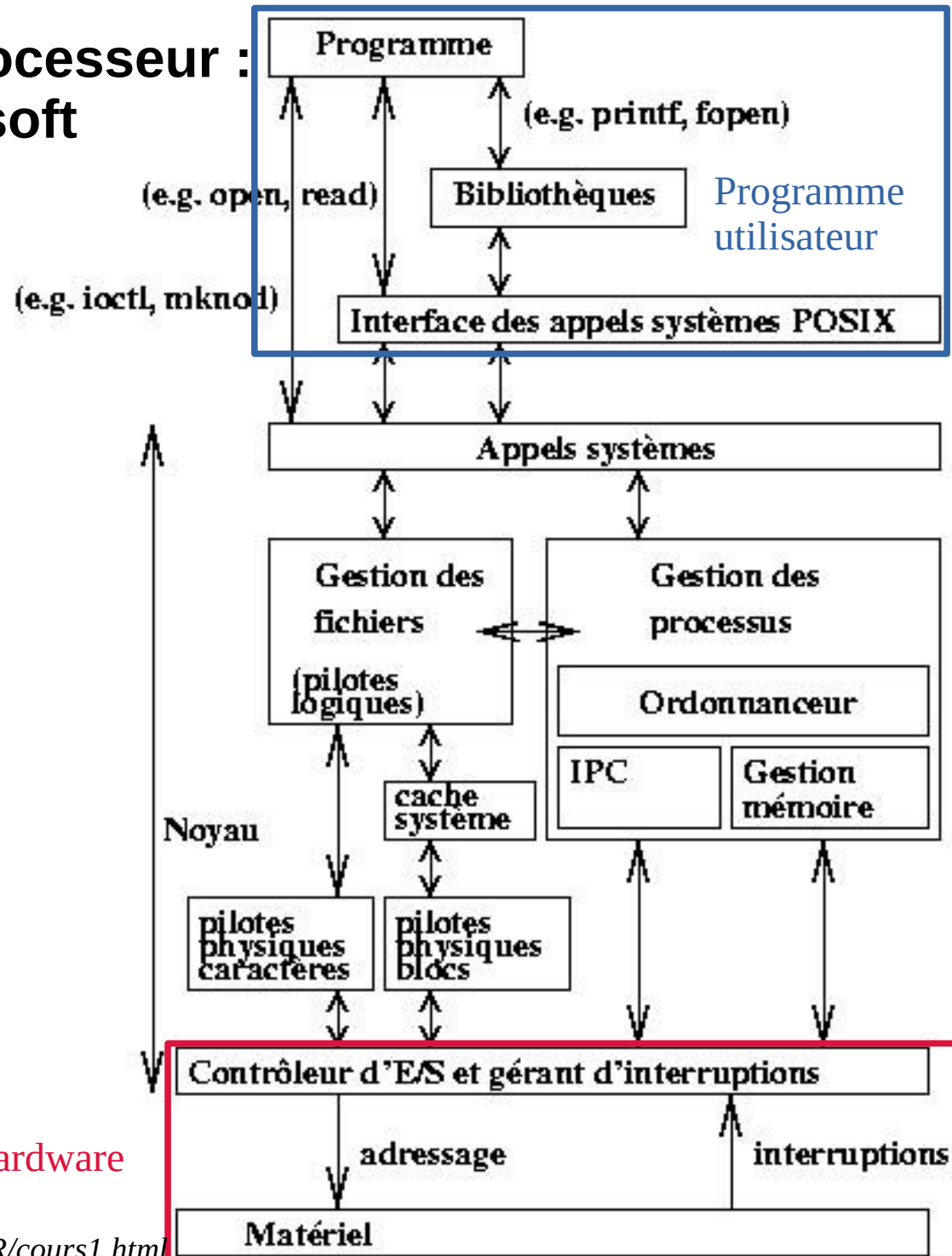
1) Processeur et mémoire(s)

Fonctions du processeur : Interfaces hard-soft

Le processeur est le chef d'orchestre, toute action est lancée par celui-ci (pour une architecture standard).

La réalisation de ces actions est déléguée à des contrôleurs pour interagir avec les autres composants hardware.

Le système d'exploitation (Linux, Windows, OSX) masque cette complexité à l'utilisateur.



Source :

<https://perso.univ-perp.fr/bernard.goossens/Ens/Lic/L3/SR/cours1.html>

Temps d'accès à la mémoire

		Access time	Capacity	Managed By
On the datapath	Registers	1 cycle	1 KB	Software/Compiler
	Level 1 Cache	2-4 cycles	32 KB	Hardware
	Level 2 Cache	10 cycles	256 KB	Hardware
On chip	Level 3 Cache	40 cycles	10 MB	Hardware
Other chips	Main Memory	200 cycles	10 GB	Software/OS
	Flash Drive	10-100us	100 GB	Software/OS
Mechanical devices	Hard Disk	10ms	1 TB	Software/OS

Source : <https://computationstructures.org/lectures/caches/caches.html>

2) Processus et thread

Processus

Wikipedia :

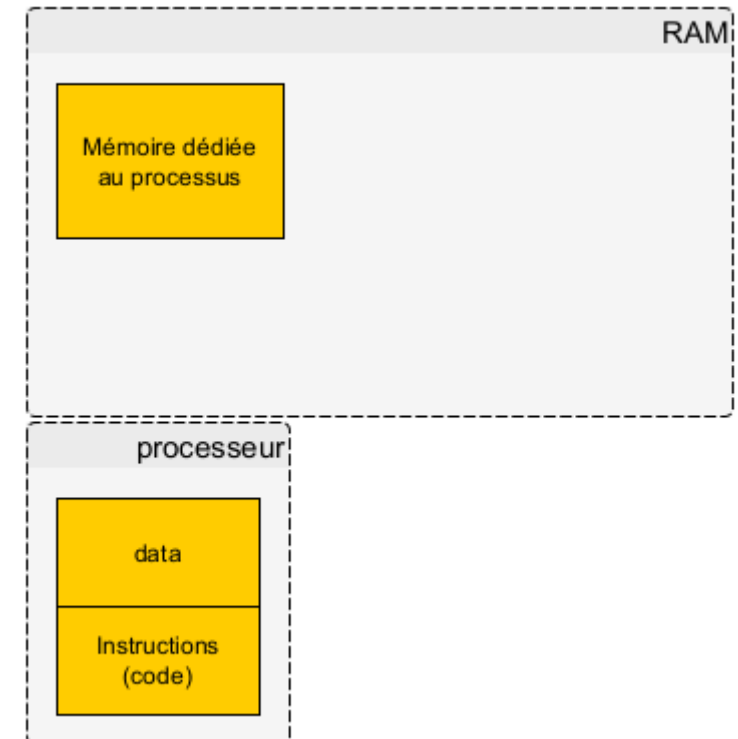
« Un processus est un programme en cours d'exécution par un ordinateur. De façon plus précise, il peut être défini comme :

- un ensemble d'instructions à exécuter, pouvant être dans la mémoire morte, mais le plus souvent chargé depuis la mémoire de masse vers la mémoire vive ;
- un espace d'adressage en mémoire vive pour stocker la pile, les données de travail, etc. ;
- des ressources permettant des entrées-sorties de données, comme des ports réseau.

L'exécution d'un processus dure un certain temps, avec un début et (parfois) une fin. Un processus peut être démarré par un utilisateur par l'intermédiaire d'un périphérique ou bien par un autre processus : les « applications » utilisateur sont des ensembles de processus. »

Le système d'exploitation (Linux) garantit l'étanchéité des espaces mémoire utilisés par différents processus.

Les processus doivent se partager les ressources physiques du nœud (mémoire, processeur, carte réseau, etc).



TP : Processus

PID (processus identifier) : identifiant unique

PPID (parent PID) : PID du processus qui a lancé ce processus

➤ /usr/bin/ps

PID	PPID	TTY	UID	STIME	COMMAND
1665	1589	pty0	1002	14:01:07	/usr/bin/ps
1589	1588	pty0	1002	13:46:35	/usr/bin/bash

PID de la commande ps

PID du terminal

➤ echo \$\$

1589

➤ sleep 10 &

[1] 1666

➤ /usr/bin/ps

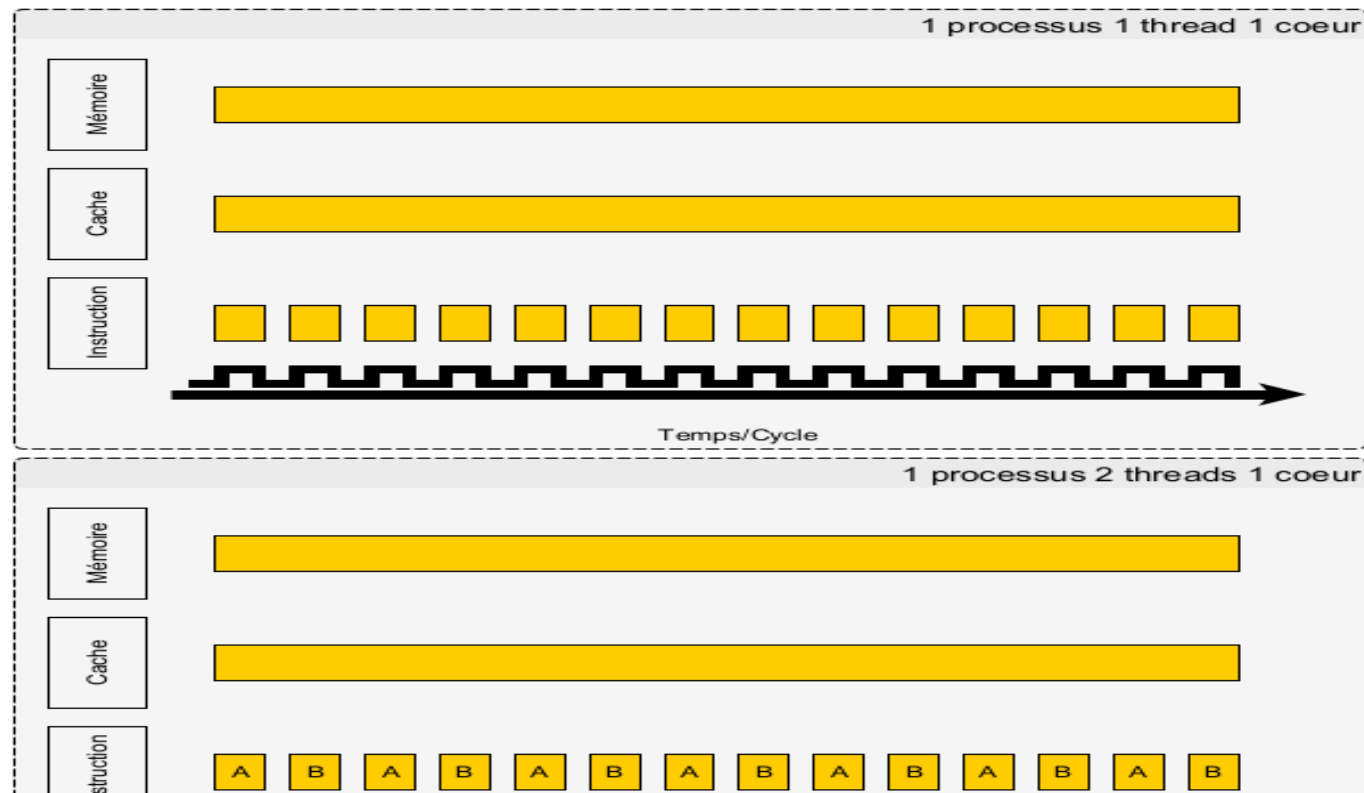
PID	PPID	TTY	UID	STIME	COMMAND
1668	1589	pty0	1002	14:08:35	/usr/bin/ps
1666	1589	pty0	1002	14:08:34	sleep 10
1589	1588	pty0	1002	13:46:35	/usr/bin/bash

Processus (2)

Un processus peut avoir 3 états : ready, running, waiting.

L'ensemble des processus tournant sur une machine se partagent le processeur.
La fréquence du processeur (en GHz) définit le nombre d'instruction qui peut exécuter par seconde.

Un processus est « running » si ses instructions s'exécutent sur le processeur.

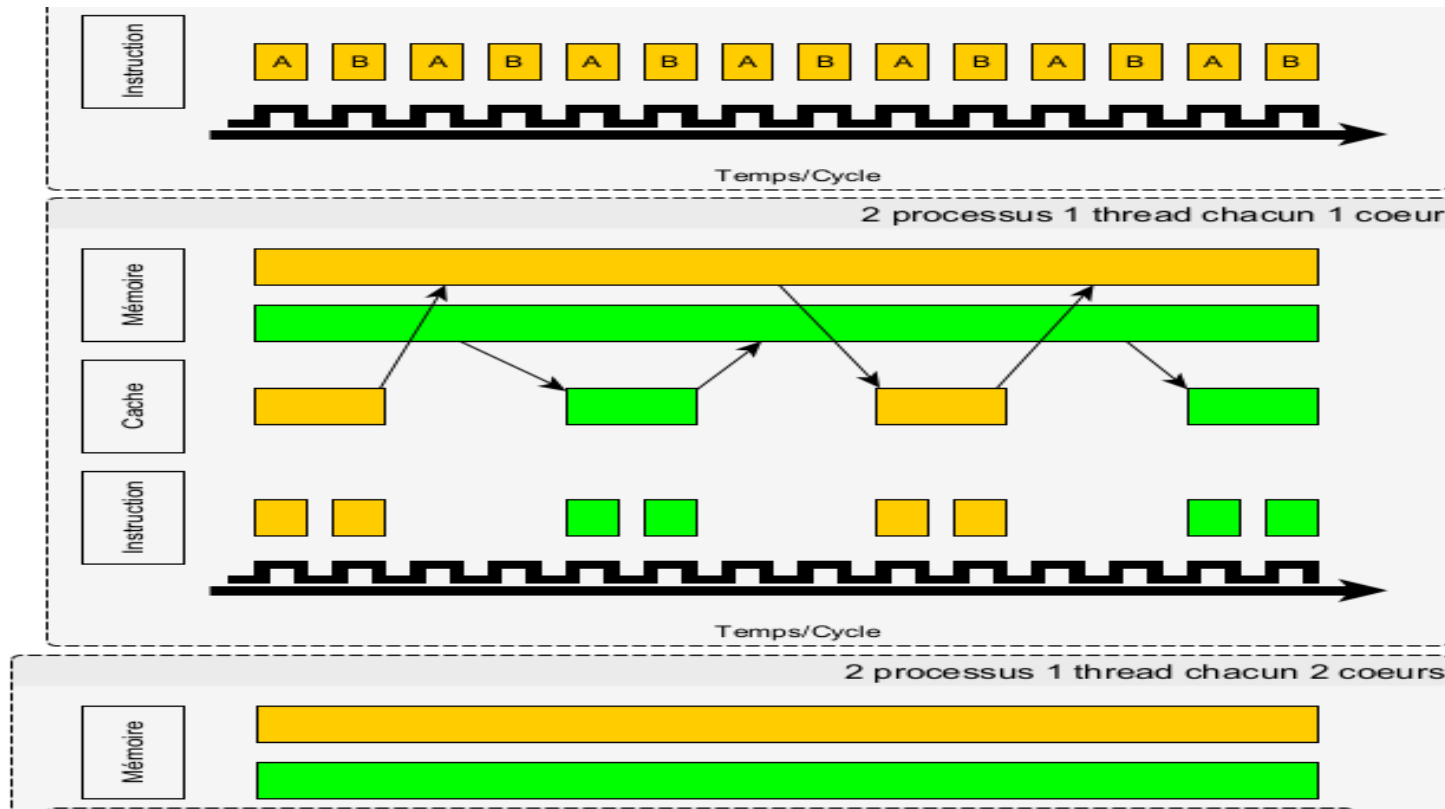


Multi-processus sur un seul cœur

Les processus sont en concurrence pour s'exécuter sur le processeur.

Lorsqu'un processus utilise le processeur, il est nécessaire de charger son « contexte » au niveau des registres et des caches.

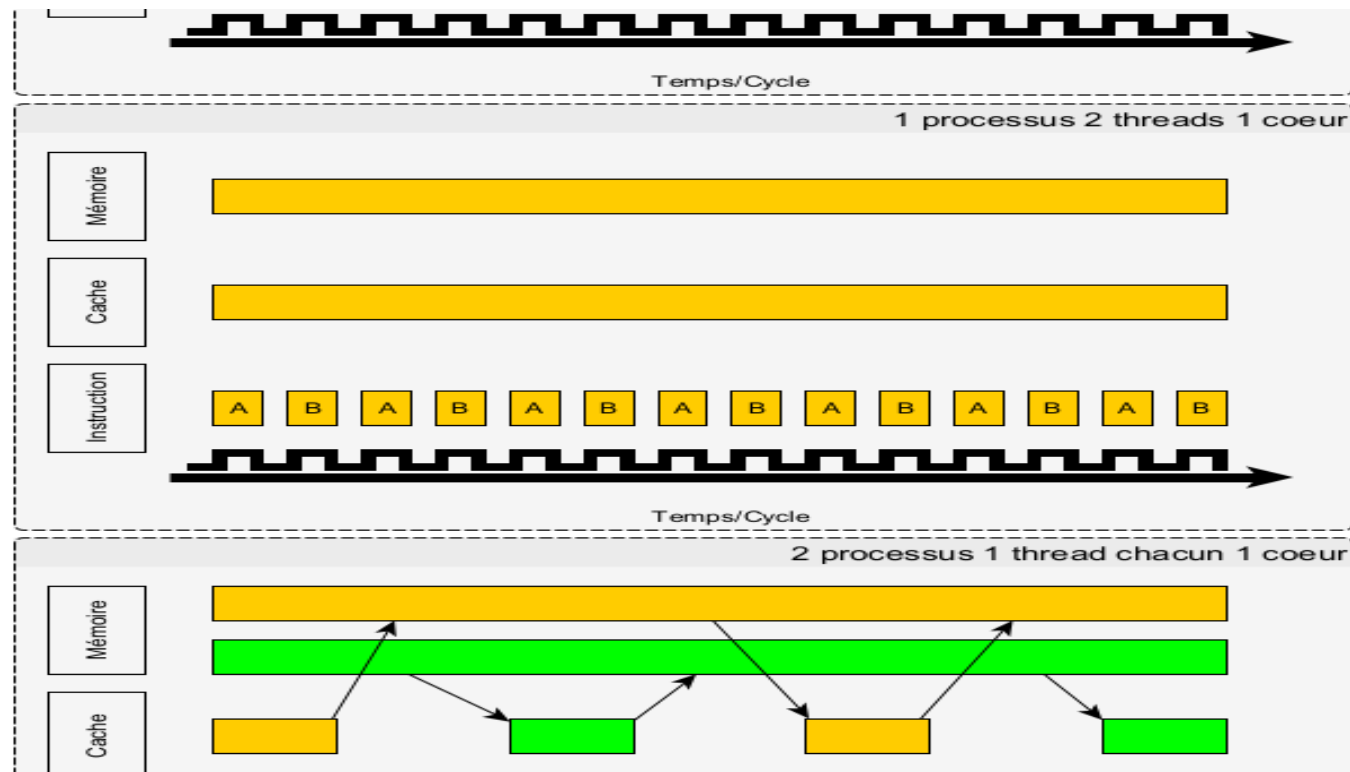
Ces aller-retours avec la mémoire font perdre des cycles. **La parallélisation multi-processus peut réduire les performances !**



Thread

Les instructions des threads sont différentes au sein d'un même processus, la mémoire est partagée (le cache pas forcément).

Pour des instructions ne nécessitant que peu de mémoire cache dédiée, c'est une manière efficace de paralléliser en consommant peu de ressources.



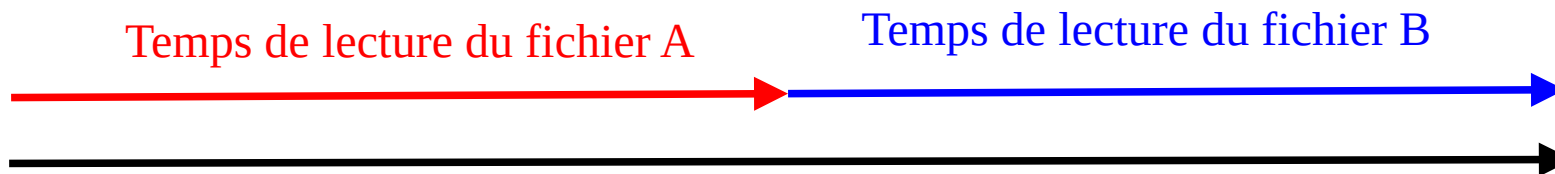
3) Parallélisation mono-nœud

Utilisation des threads pour la parallélisation

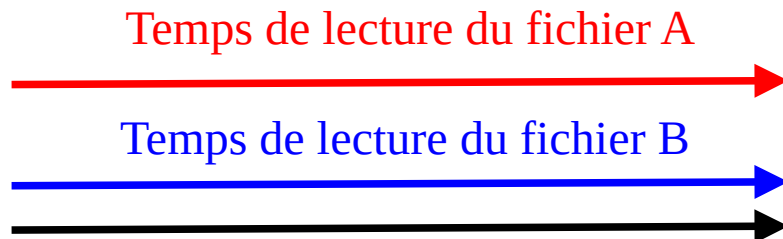
Les threads en mono-coeur permettent de paralléliser du temps d'attente (ouverture de fichier, téléchargement, etc).

Lors de ce type d'action, les threads ont principalement l'état « waiting » et n'occupent de pas de cycles.

Traitement séquentiel :



Traitement parallèle avec 2 threads :



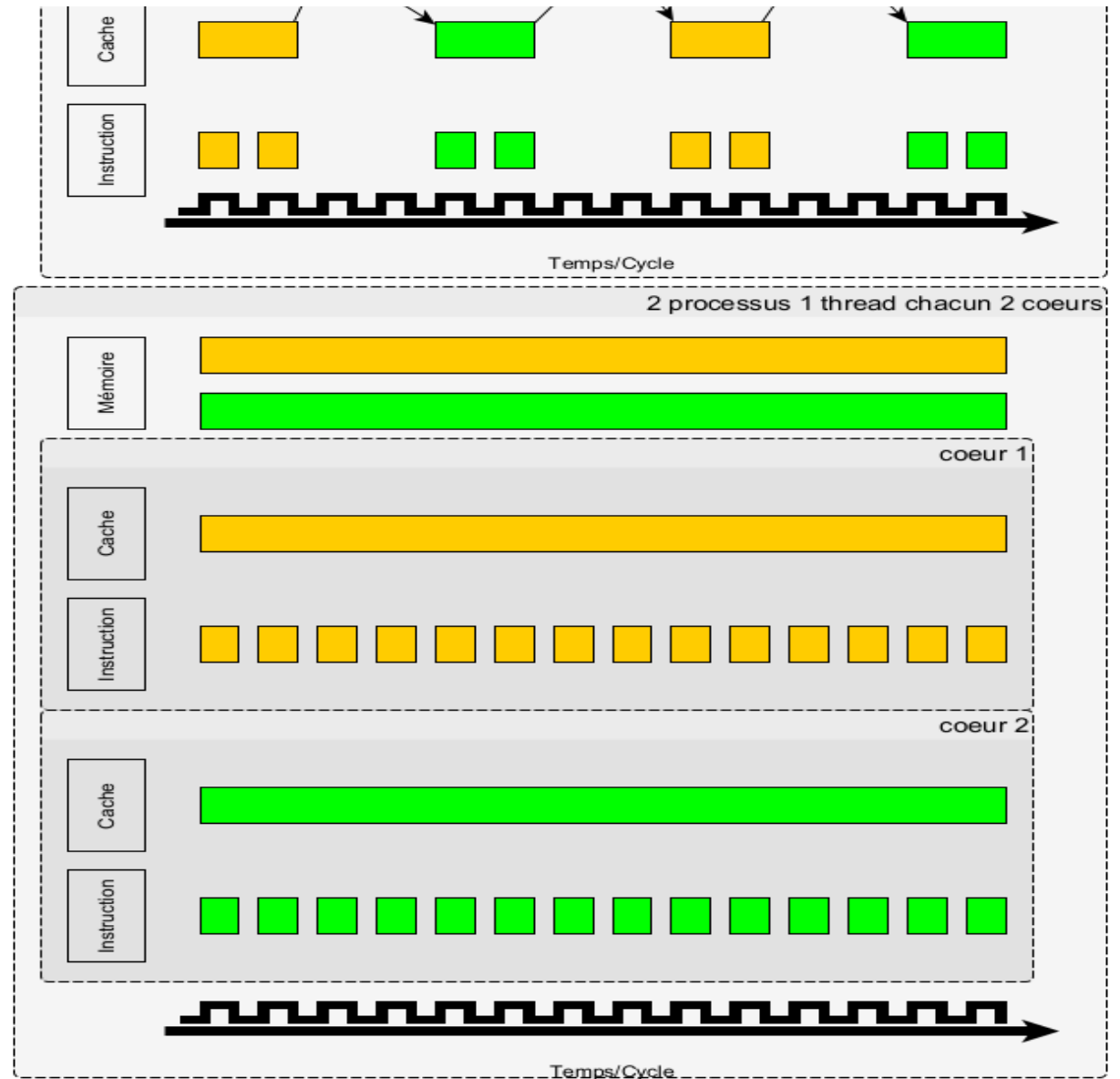
(sans tenir compte des performances du stockage)

Multi-processus sur deux cœurs

Avec deux cœurs chaque processus occupe un cœur, il n'y a plus de concurrence et de perte de cycle.

La parallélisation est efficace, pour des processus indépendants.

Il faut disposer d'assez de mémoire RAM pour chaque processus. Sinon il y a des risques d'« Out Of Memory » (OOM) qui entraîne un redémarrage du nœud, ou de l'utilisation du disque dur pour stocker temporairement la mémoire (swap).



Comment lancer des processus en parallèle ?

TP : background.sh et boucle_sleep.sh

➤ *time ./boucle_sleep.sh*

real 0m 5.27s

user 0m 0.04s

sys 0m 0.10s

➤ *time ./background.sh*

real 0m 5.31s

user 0m 0.03s

sys 0m 0.16s

4) Communication entre processus et MPI

Communication inter-processus

Par construction, un processus A ne peut pas lire ou écrire dans la mémoire du processus B. Les calculs réalisés par A et B doivent être indépendants ou échanger des données par des moyens non efficaces (écriture/lecture de fichier).

La solution pour échanger des données entre processus directement en mémoire est de définir des emplacements mémoires spécifiques accessibles en écriture entre processus : des « queues de messages ».

Les « messages » permettent de copier des données d'un emplacement mémoire d'un processus à la mémoire d'un autre processus. Ces messages sont reçus dans le buffer du processus qui le reçoit.

La librairie Message Passing Interface (MPI) permet de masquer les mécanismes de bas niveaux.

MPI : communicateur et rangs

Le lancement des différents processus de traitement avec MPI se fait via un appel à la fonction « MPI_INIT ». Cet appel définit aussi un rang (entier) pour identifier chaque processus ainsi que les buffers pour échanger les messages.

Le rang « 0 » est le processus initial, les rangs sont ensuite définis arbitrairement ou via des arguments pour « placer » les processus.

Le rang est ce qui permet de particulariser le comportement d'un processus par rapport aux autres. Le code chargé par l'ensemble des processus est identique, il faut donc instrumenter le code avec des « if rank == i » pour particulariser les traitements de chaque processus.

Avoir plusieurs processus exécutant le même code sur les mêmes données n'a pas un grand intérêt.

MPI : send et receive

Exemple de code où pour envoyer le contenu de 2 variables varA et varB du processus 0 au processus 1 :

```

    Taille en octets
    If (my_rank == 0) {
        MPI_Send(varA, 10, MPI_INT, 1, id1, MPI_COMM_WORLD)
        MPI_Send(varB, 10, MPI_INT, 1, id2, MPI_COMM_WORLD)
    }
    Type de la variable
    Identifiant du message
    (pour les différencier)
    Else if (my_rank == 1) {
        MPI_Recv(varA, 10, MPI_INT, 0, id1, MPI_COMM_WORLD, status_error)
        MPI_Recv(varB, 10, MPI_INT, 0, id2, MPI_COMM_WORLD, status_error)
    }
    Rang de l'émetteur
    Rang du destinataire
    Communicateur à utiliser

```

Code exécuté sur le processus 0

Code exécuté sur le processus 1

Dans cet exemple les communications sont bloquantes, le processus « 1 » attend d'avoir reçu le message « id1 » avant de lire « id2 ».

Pour avoir des communications non bloquantes, il faut utiliser les méthodes « isend » et « irecv ». Dans ce cas il peut être nécessaire d'utiliser la méthode MPI_Barrier pour forcer la synchronisation de tous les processus.

5) Optimiser avant de paralléliser

Golden rule

La parallélisation, surtout en multi-processus, est l'artillerie lourde en matière de consommation de ressources de calcul.

Avant de se lancer dans la parallélisation, il faut toujours commencer par optimiser le code séquentiel.

Dans certain cas utiliser la « bonne » fonction ou librairie Python peut être plus rapide que de paralléliser (cf exemple du TP sur les moyennes).

L'optimisation est un vaste sujet en soit qui ne sera pas traité dans ce cours.